

6.3 Navigation Architectures

Given techniques for path planning, obstacle avoidance, localization, and perceptual interpretation, how do we combine all of these into one complete robot system for a real-world application? One way to proceed would be to custom-design an application-specific, monolithic software system that implements everything for a specific purpose. This may be efficient in the case of a trivial mobile robot application with few features and even fewer planned demonstrations. But for any sophisticated and long-term mobile robot system, the issue of mobility architecture should be addressed in a principled manner. The study of *navigation architectures* is the study of principled designs for the software modules that constitute a mobile robot navigation system. Using a well-designed navigation architecture has a number of concrete advantages:

6.3.1 Modularity for code reuse and sharing

Basic software engineering principles embrace software modularity, and the same general motivations apply equally to mobile robot applications. But modularity is of even greater importance in mobile robotics because in the course of a single project the mobile robot hardware or its physical environmental characteristics can change dramatically, a challenge most traditional computers do not face. For example, one may introduce a Sick laser rangefinder to a robot that previously used only ultrasonic rangefinders. Or one may test an existing navigator robot in a new environment where there are obstacles that its sensors cannot detect, thereby demanding a new path-planning representation.

We would like to change part of the robot's competence without causing a string of side effects that force us to revisit the functioning of other robot competences. For instance we would like to retain the obstacle avoidance module intact, even as the particular ranging sensor suite changes. In a more extreme example, it would be ideal if the nonholonomic obstacle avoidance module could remain untouched even when the robot's kinematic structure changes from a tricycle chassis to a differential-drive chassis.

6.3.2 Control localization

Localization of robot control is an even more critical issue in mobile robot navigation. The basic reason is that a robot architecture includes multiple types of control functionality (e.g., obstacle avoidance, path planning, path execution, etc.). By localizing each functionality to a specific unit in the architecture, we enable individual testing as well as a principled strategy for control composition. For example, consider collision avoidance. For stability in the face of changing robot software, as well as for focused verification that the obstacle avoidance system is correctly implemented, it is valuable to localize all software related to the robot's obstacle avoidance process. At the other extreme, high-level planning and task-based decision-making are required for robots to perform useful roles in their

environment. It is also valuable to localize such high-level decision-making software, enabling it to be tested exhaustively in simulation and thus verified even without a direct connection to the physical robot. A final advantage of localization is associated with learning. Localization of control can enable a specific learning algorithm to be applied to just one aspect of a mobile robot's overall control system. Such targeted learning is likely to be the first strategy that yields successful integration of learning and traditional mobile robotics.

The advantages of localization and modularity prove a compelling case for the use of principled navigation architectures.

One way to characterize a particular architecture is by its decomposition of the robot's software. There are many favorite robot architectures, especially when one considers the relationship between artificial intelligence level decision making and lower-level robot control. For descriptions of such high-level architectures, refer to [2] and [26]. Here we concentrate on navigation competence. For this purpose, two decompositions are particularly relevant: temporal decomposition and control decomposition. In section 6.3.3 we define these two types of decomposition, then present an introduction to *behaviors*, which are a general tool for implementing control decomposition. Then, in section 6.3.4 we present three types of navigation architectures, describing for each architecture an implemented mobile robot case study.

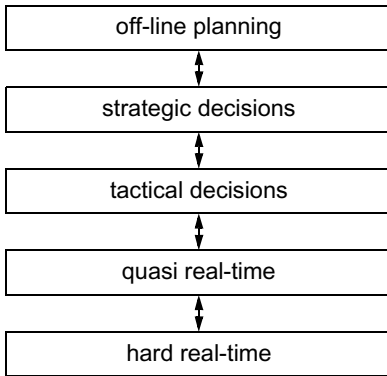
6.3.3 Techniques for decomposition

Decompositions identify axes along which we can justify discrimination of robot software into distinct modules. Decompositions also serve as a way to classify various mobile robots into a more quantitative taxonomy. *Temporal decomposition* distinguishes between real-time and non real-time demands on mobile robot operation. *Control decomposition* identifies the way in which various control outputs within the mobile robot architecture combine to yield the mobile robot's physical actions. Below we describe each type of decomposition in greater detail.

6.3.3.1 Temporal decomposition

A temporal decomposition of robot software distinguishes between processes that have varying real-time and non real-time demands. Figure 6.18 depicts a generic temporal decomposition for navigation. In this figure, the most real-time processes are shown at the bottom of the *stack*, with the highest category being occupied by processes with no real-time demands.

The lowest level in this example captures functionality that must proceed with a guaranteed fast cycle time, such as a 40 Hz bandwidth. In contrast, a quasi real-time layer may capture processes that require, for example, 0.1 second response time, with large allowable worst-case individual cycle times. A tactical layer can represent decision-making that

**Figure 6.18**

Generic temporal decomposition of a navigation architecture.

affects the robot's immediate actions and is therefore subject to some temporal constraints, while a strategic or off-line layer represents decisions that affect the robot's behavior over the long term, with few temporal constraints on the module's response time.

Four important, interrelated trends correlate with temporal decomposition. These are not set in stone; there are exceptions. Nevertheless, these general properties of temporal decompositions are enlightening:

Sensor response time. A particular module's sensor response time can be defined as the amount of time between acquisition of a sensor-based event and a corresponding change in the output of the module. As one moves up the stack in figure 6.18 the sensor response time tends to increase. For the lowest-level modules, the sensor response time is often limited only by the raw processor and sensor speeds. At the highest-level modules, sensor response can be limited by slow and deliberate decision-making processes.

Temporal depth. Temporal depth is a useful concept applying to the temporal window that affects the module's output, both backward and forward in time. *Temporal horizon* describes the amount of look ahead used by the module during the process of choosing an output. *Temporal memory* describes the historical time span of sensor input that is used by the module to determine the next output. Lowest-level modules tend to have very little temporal depth in both directions, whereas the deliberative processes of highest-level modules make use of a large temporal memory and consider actions based on their long-term consequences, making note of large temporal horizons.

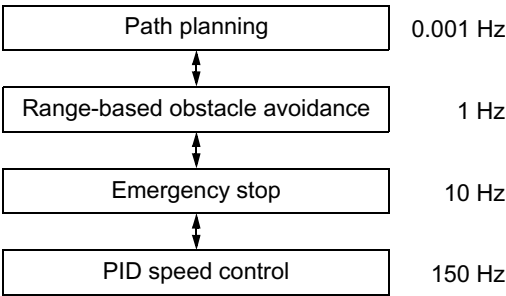
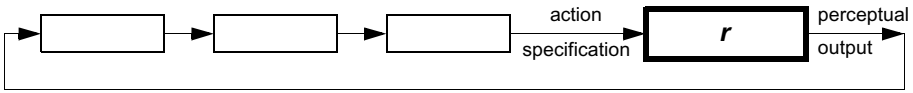


Figure 6.19
Sample four-level temporal decomposition of a simple navigating mobile robot. The column on the right indicates realistic bandwidth values for each module.

Spatial locality. Hand in hand with temporal span, the spatial impact of layers increases dramatically as one moves from low-level modules to high-level modules. Real-time modules tend to control wheel speed and orientation, controlling spatially localized behavior. High-level strategic decision-making has little or no bearing on local position, but informs global position far into the future.

Context specificity. A module makes decisions as a function not only of its immediate inputs but also as a function of the robot’s context as captured by other variables, such as the robot’s representation of the environment. Lowest-level modules tend to produce outputs directly as a result of immediate sensor inputs, using little context and therefore being relatively context insensitive. Highest-level modules tend to exhibit very high context specificity. For strategic decision-making, given the same sensor values, dramatically different outputs are nevertheless conceivable depending on other contextual parameters.

An example demonstrating these trends is depicted in figure 6.19, which shows a temporal decomposition of a simplistic navigation architecture into four modules. At the lowest level the PID control loop provides feedback to control motor speeds. An emergency stop module uses short-range optical sensors and bumpers to cut current to the motors when it predicts an imminent collision. Knowledge of robot dynamics means that this module by nature has a greater temporal horizon than the PID module. The next module uses longer-range laser rangefinding sensor returns to identify obstacles well ahead of the robot and make minor course deviations. Finally, the path planner module takes the robot’s initial and goal positions and produces an initial trajectory for execution, subject to change based on actual obstacles that the robot collects along the way.

**Figure 6.20**

Example of a pure serial decomposition.

Note that the cycle time, or bandwidth, of the modules changes by orders of magnitude between adjacent modules. Such dramatic differences are common in real navigation architectures, and so temporal decomposition tends to capture a significant axis of variation in a mobile robot's navigation architecture.

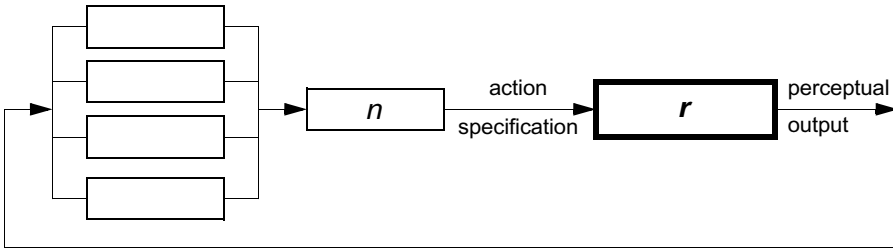
6.3.3.2 Control decomposition

Whereas temporal decomposition discriminates based on the time behavior of software modules, control decomposition identifies the way in which each module's output contributes to the overall robot control outputs. Presentation of control decomposition requires the evaluator to understand the basic principles of discrete systems representation and analysis. For a lucid introduction to the theory and formalism of discrete systems, see [17, 71].

Consider the robot algorithm and the physical robot instantiation (i.e., the robot form and its environment) to be members of an overall system whose connectivity we wish to examine. This overall system S is comprised of a set M of modules, each module m connected to other modules via inputs and outputs. The system is *closed*, meaning that the input of every module m is the output of one or more modules in M . Each module has precisely one output and one or more inputs. The one output can be connected to any number of other modules inputs.

We further name a special module r in M to represent the physical robot and environment. Usually by r we represent the physical object on which the robot algorithm is intended to have impact, and from which the robot algorithm derives perceptual inputs. The module r contains one input and one output line. The input of r represents the complete action specification for the physical robot. The output of r represents the complete perceptual output to the robot. Of course the physical robot may have many possible degrees of freedom and, equivalently, many discrete sensors. But for this analysis we simply imagine the entire input/output vector, thus simplifying r to just one input and one output. For simplicity we will refer to the input of r as O and to the robot's sensor readings I . From the point of view of the rest of the control system, the robot's sensor values I are inputs, and the robot's actions O are the outputs, explaining our choice of I and O .

Control decomposition discriminates between different types of control pathways through the portion of this system comprising the robot algorithm. At one extreme, depicted in figure 6.20 we can consider a perfectly linear, or sequential control pathway.

**Figure 6.21**

Example of a pure parallel decomposition.

Such a serial system uses the internal state of all associated modules and the value of the robot's percept I in a sequential manner to compute the next robot action O . A pure serial architecture has advantages relating to predictability and verifiability. Since the state and outputs of each module depend entirely on the inputs it receives from the module upstream, the entire system, including the robot, is a single well-formed loop. Therefore, the overall behavior of the system can be evaluated using well-known discrete forward simulation methods.

Figure 6.21 depicts the extreme opposite of pure serial control, a fully parallel control architecture. Because we choose to define r as a module with precisely one input, this parallel system includes a special module n that provides a single output for the consumption of r . Intuitively, the fully parallel system distributes responsibility for the system's control output O across multiple modules, possibly simultaneously. In a pure sequential system, the control flow is a linear sequence through a string of modules. Here, the control flow contains a *combination* step at which point the result of multiple modules may impact O in arbitrary ways.

Thus parallelization of control leads to an important question: how will the output of each component module inform the overall decision concerning the value of O ? One simple combination technique is temporal switching. In this case, called *switched parallel*, the system has a parallel decomposition but at any particular instant in time the output O can be attributed to one specific module. The value of O can of course depend on a different module at each successive time instant, but the instantaneous value of O can always be determined based on the functions of a single module. For instance, suppose that a robot has an obstacle avoidance module and a path-following module. One switched control implementation may involve execution of the path-following recommendation whenever the robot is more than 50 cm from all sensed obstacles and execution of the obstacle avoidance recommendation when any sensor reports a range closer than 50 cm.

The advantage of such switched control is particularly clear if switching is relatively rare. If the behavior of each module is well understood, then it is easy to characterize the behavior of the switched control robot: it will obstacle avoid at times, and it will path-follow other times. If each module has been tested independently, there is a good chance the switched control system will also perform well. Two important disadvantages must be noted. First, the overall behavior of the robot can become quite poor if the switching is itself a high-frequency event. The robot may be unstable in such cases, switching motion modes so rapidly as to dramatically devolve into behavior that is neither path-following nor obstacle avoiding. Another disadvantage of switched control is that the robot has no path-following bias when it is obstacle avoiding (and vice versa). Thus in cases where control *ought* to mix recommendations from among multiple modules, the switched control methodology fails.

In contrast, the much more complex *mixed parallel* model allows control at any given time to be shared between multiple modules. For example, the same robot could take the obstacle avoidance module's output at all times, convert it to a velocity vector, and combine it with the path-following module's output using vector addition. Then the output of the robot would never be due to a single module, but would result from the mathematical combination of both modules outputs. Mixed parallel control is more general than switched control, but by that token it is also a more challenging technique to use well. Whereas with switched control most poor behavior arises out of inopportune switching behavior, in mixed control the robot's behavior can be quite poor even more readily. Combining multiple recommendations mathematically does not guarantee an outcome that is globally superior, just as combining multiple vectors when deciding on a swerve direction to avoid an obstacle can result in the very poor decision of straight ahead. Thus great care must be taken in mixed parallel control implementations to fashion mixture formulas and individual module specifications that lead to effective mixed results.

Both the switched and mixed parallel architectures are popular in the behavior-based robotics community. Arkin [2] proposes the *motor-schema* architecture in which *behaviors* (i.e., modules in the above discussion) map sensor value vectors to motor value vectors. The output of the robot algorithm is generated, as in mixed parallel systems, using a linear combination of the individual behavior outputs. In contrast, Maes [103, 104] produces a switched parallel architecture by creating a *behavior network* in which a behavior is chosen discretely by comparing and updating activation levels for each behavior. The subsumption architecture of Brooks [45] is another example of a switched parallel architecture, although the active model is chosen via a suppression mechanism rather than activation level. For a further discussion, see [2].

One overall disadvantage of parallel control is that verification of robot performance can be extremely difficult. Because such systems often include truly parallel, multithreaded implementations, the intricacies of robot-environment interaction and sensor timing

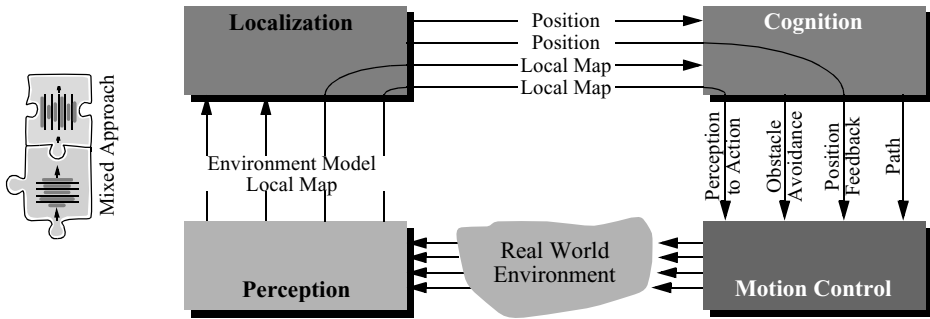


Figure 6.22

The basic architectural example used throughout this text.

required to properly represent all conceivable module-module interactions can be difficult or impossible to simulate. So, much testing in the parallel control community is performed empirically using physical robots.

An important advantage of parallel control is its biomimetic aspect. Complex organic organisms benefit from large degrees of true parallelism (e.g., the human eye), and one goal of the parallel control community is to understand this biologically common strategy and leverage it to advantage in robotics.

6.3.4 Case studies: tiered robot architectures

We have described temporal and control decompositions of robot architecture, with the common theme that the roboticist is always composing multiple modules together to make up that architecture. Let us turn again toward the overall mobile robot navigation task with this understanding in mind. Clearly, robot behaviors play an important role at the real-time levels of robot control, for example, path-following and obstacle avoidance. At higher temporal levels, more tactical tasks need to modulate the activation of behaviors, or modules, in order to achieve robot motion along the intended path. Higher still, a global planner could generate paths to provide tactical tasks with global foresight.

In chapter 1, we introduced a functional decomposition showing such modules of a mobile robot navigator from the perspective of information flow. The relevant figure is shown here again as figure 6.22.

In such a representation, the arcs represent aspects of real-time and non real-time competence. For instance, obstacle avoidance requires little input from the localization module and consists of fast decisions at the cognition level followed by execution in motion control. In contrast, PID position feedback loops bypass all high-level processing, tying the perception of encoder values directly to lowest-level PID control loops in motion control.